

1. Reorder a Singly linked list.

Given linked List.

$L_1 \rightarrow L_2 \rightarrow L_3 \rightarrow \dots \rightarrow L_{n-1} \rightarrow L_n$

The list should be reordered as:

$L_1 \rightarrow L_n \rightarrow L_2 \rightarrow L_{n-1} \rightarrow L_3 \rightarrow L_{n-2} \rightarrow \dots$

Example :

Original List:

$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$

Reordered List:

$1 \rightarrow 6 \rightarrow 2 \rightarrow 5 \rightarrow 3 \rightarrow 4$

Steps involved.

1. Traverse the linked list to find the middle node.
2. Reverse the second half of the linked list.
3. Merge the first half and reversed second half alternately.
4. Print the reordered linked list.

Explanation:

The reordering process starts with the first node, followed by the last node, the second node, then the second-last node and so on until all nodes are

rearranged. This method does not create new nodes; instead, it changes the links

2. Doubly Linked List Operations.

Given Elements.

150, 250, 350, 450, 550.

Step 1: Insert the Element.

After inserting all elements the doubly linked list becomes:

$NULL \leftrightarrow 150 \leftrightarrow 250 \leftrightarrow 350 \leftrightarrow 450 \leftrightarrow$

$550 \leftrightarrow NULL.$

Step 2: Delete 150.

Since 150 is the first Node (Head), update the head pointer to the next node.

$NULL \leftrightarrow 250 \leftrightarrow 350 \leftrightarrow 450 \leftrightarrow 550 \leftrightarrow NULL.$

Step 3: Delete 250.

Now, 250 becomes the first node. Delete it by updating the head pointer.

$NULL \leftrightarrow 350 \leftrightarrow 450 \leftrightarrow 550 \leftrightarrow NULL.$

Final Doubly Linked List.

$\text{NULL} \leftarrow \rightarrow 350 \leftarrow \rightarrow 450 \leftarrow \rightarrow 550 \leftarrow \rightarrow \text{NULL}.$

Explanation:

A doubly linked list consists of nodes containing three fields: a previous pointer, data and a next pointer. During insertion, each new node is connected to the previous node using both forward and backward links. When deleting the head node (150), the head pointer is updated to the next node (250), and its previous pointer is set to NULL. Similarly deleting 250 updates the head to 350. Because each node stores both previous and next pointers, insertion and deletion operations can be performed efficiently without traversing the entire list.